

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO5 ASSESSMENT TOOL 2 – Seminar on Emerging Technologies

Course Code:	CSA10 / CSA1002	Course Title:	Software Engineering
CO Assessed:	CO5	Assessment:	Assessment Tool 2 – Seminar on Mutation Testing
Weightage:	20%	Total Marks:	25
CO5 Statement:	Evaluate emerging technologies and societal impacts, maintaining and evolving software systems responsibly		
Student Name:	RAVURU PRANATHI	Reg. No.:	192525076
Date:	14-08-2026	Duration:	60 minutes

Code of Conduct

I R.Pranathi(192525076) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *R. Pranathi*

Annexure A – Seminar on Emerging Technologies

Instructions to Students:

Students must select the topic strictly according to the Serial Number (S.No.) order assigned to them.

S.No.	Seminar Topic
1	Generative AI and Large Language Models (LLMs)
2	AI Agents and Autonomous AI Systems
3	Multimodal Artificial Intelligence
4	Explainable and Trustworthy AI
5	Edge AI and Intelligent Edge Computing
6	Federated Learning and Privacy-Preserving AI
7	Deep Learning for Computer Vision
8	AI-Powered Medical Image Analysis
9	Natural Language Processing with Transformer Models
10	Retrieval-Augmented Generation (RAG)
11	AI in Cybersecurity and Threat Detection
12	Zero Trust Architecture and Modern Cybersecurity
13	Post-Quantum Cryptography
14	Quantum Computing and Quantum Algorithms
15	Quantum Machine Learning
16	Cloud-Native Computing and Microservices
17	Serverless Computing and Function-as-a-Service
18	Edge Computing and Fog Computing
19	Cloud Security and Confidential Computing
20	Digital Twins and Intelligent Simulation

21	Internet of Things (IoT) and Smart Systems
22	Industrial Internet of Things (IIoT)
23	5G/6G Networks and Next-Generation Communication
24	Blockchain Technology and Decentralized Applications
25	Web3 and Decentralized Internet
26	Smart Contracts and Blockchain Applications
27	Augmented Reality (AR) and Virtual Reality (VR)
28	Extended Reality (XR) and Immersive Technologies
29	Robotics and Autonomous Robots
30	Human–Robot Interaction and Collaborative Robots
31	Autonomous Vehicles and Intelligent Transportation Systems
32	Smart Cities and AI-Driven Urban Technologies
33	Metaverse Technologies and Virtual Worlds
34	Brain–Computer Interfaces (BCI)
35	Wearable Computing and Smart Wearable Devices
36	Green Computing and Sustainable IT
37	AI for Software Engineering and Code Generation
38	DevSecOps and Intelligent CI/CD Pipelines
39	Digital Forensics and AI-Based Cybercrime Investigation
40	Privacy-Enhancing Technologies and Data Protection

Seminar Presentation Structure:

1. **Introduction to the Emerging Technology**
2. **Need and Motivation**
3. **Working Principle / Architecture**
4. **Key Components and Technologies**
5. **Applications / Real-World Use Cases**
6. **Advantages and Limitations**
7. **Recent Developments and Trends**
8. **Challenges and Future Scope**
9. **Case Study / Example**
10. **Conclusion and References**

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Technical Content & Subject	Comprehensive and accurate content; demonstrates excellent	Good technical understanding with minor gaps	Basic understanding with limited technical depth	Incomplete content or major technical errors

Knowledge (25%)	understanding of the emerging technology			
Organization & Presentation Structure (15%)	Excellent logical flow with clear introduction, content, conclusion, and effective transitions	Well-structured with minor organizational issues	Adequately structured but some sections lack clarity	Poorly structured and difficult to follow
Examples, Diagrams & Case Study (20%)	Excellent use of relevant diagrams, examples, applications, and real-world case studies	Good use of relevant examples and diagrams	Limited examples/diagrams with basic explanation	Inappropriate or insufficient examples/diagrams
Communication & Presentation Skills (20%)	Very clear, confident, engaging delivery with excellent eye contact, voice modulation, and time management	Clear and confident presentation with minor issues	Understandable but lacks confidence or engagement	Unclear delivery, excessive reading, or poor time management
Question Handling & Overall Performance (20%)	Answers questions accurately and confidently; demonstrates strong overall knowledge	Answers most questions correctly	Answers basic questions with some difficulty	Unable to answer questions or demonstrate adequate understanding

Criteria	Mark
Technical Content & Subject Knowledge (25%)	/5
Organization & Presentation Structure (15%)	/5
Examples, Diagrams & Case Study (20%)	/5
Communication & Presentation Skills (20%)	/5
Question Handling & Overall Performance (20%)	/5
TOTAL	/25

MUTATION TESTING IN SOFTWARE ENGINEERING

1. Introduction

Software is used in almost every field such as banking, healthcare, education, transportation, e-commerce, communication, and entertainment. As software becomes more important, it is necessary to make sure that it works correctly and reliably.

Software testing is one of the most important activities in software development. It is used to find errors, check requirements, verify outputs, and improve software quality. However, having a large number of test cases does not always mean that the testing is effective. A test suite may execute the program successfully but still fail to detect certain types of faults.

This creates an important question:

How can we know whether our test cases are strong enough to detect errors?

Mutation testing provides an answer to this question.

Mutation testing is a software testing technique in which small changes are intentionally made to a correct program. These changed versions are called **mutants**. The existing test cases are then executed against the mutants. If a test case detects the introduced change, the mutant is called **killed**. If the test cases do not detect the change, the mutant is called **survived**.

Therefore, mutation testing can be understood as a technique for **testing the effectiveness of the test cases themselves**.

2. Objectives of Mutation Testing

The main objectives of mutation testing are:

1. To evaluate the quality of test cases.
2. To identify weak test cases.
3. To find missing test cases.
4. To check whether important conditions are properly tested.
5. To identify problems in boundary-value testing.
6. To improve the reliability of software testing.
7. To measure the effectiveness of a test suite.
8. To provide additional information beyond code coverage.

Mutation testing checks whether our test cases are good enough to find mistakes in the program.

3. What is Software Testing?

Software testing is the process of checking software to determine whether it behaves according to its requirements.

Testing normally involves:

- Providing inputs to the software.
- Executing the software.
- Observing the output.
- Comparing the actual output with the expected output.
- Identifying errors or unexpected behavior.

For example, consider a calculator.

If the user enters:

5 + 5

The expected result is:

10

If the software produces:

15

then the software contains an error.

Testing helps identify such problems before the software reaches the users.

4. Need for Mutation Testing

Traditional testing is useful, but it has a limitation.

Suppose a program contains 100 test cases and all 100 test cases pass. We may think that the test suite is very good.

However, passing tests do not always mean that the test cases are strong.

For example, suppose a program contains the condition:

if age >= 18
Eligible

The test cases may contain:

- Age = 20
- Age = 25
- Age = 30

All these tests pass.

But the important boundary value:

Age = 18

has not been tested.

Now suppose someone changes:

age $\geq$ 18

to:

age > 18

This is a small change, but it creates a different result for age 18.

If the test suite does not contain age 18, it may fail to detect this problem.

Mutation testing deliberately creates such changes and checks whether the test cases can detect them.

5. Definition of Mutation Testing

Mutation testing is a software testing technique in which small changes are intentionally introduced into a correct program to create modified versions called **mutants**.

The test suite is executed on these mutants.

There are two main outcomes:

Killed Mutant

If the test cases detect the change, the mutant is killed.

Survived Mutant

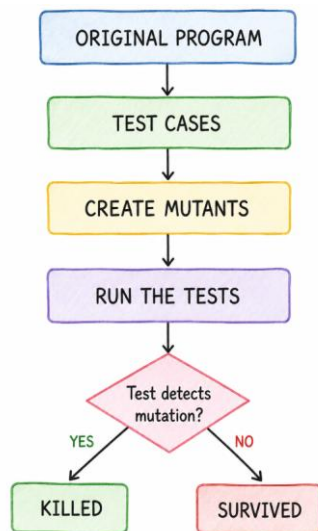
If the test cases do not detect the change, the mutant survives.

In simple words:

Mutation testing means making small changes in a program to check whether our test cases are strong enough to find those changes.

6. Basic Concept of Mutation Testing

The basic concept can be represented as:



The purpose of this process is to identify whether the test suite can detect small faults.

7. Important Terms

7.1 Original Program

The correct program before any changes are introduced is called the original program.

7.2 Mutant

A mutant is a modified version of the original program containing a small intentional change.

7.3 Mutation

The actual change introduced into the program is called a mutation.

Example:

$a + b$

changed to:

$a - b$

Here:

$+ \rightarrow -$

is the mutation.

7.4 Test Case

A test case contains inputs and expected results used to check whether the software behaves correctly.

7.5 Killed Mutant

A mutant is killed when at least one test case detects the introduced change.

7.6 Survived Mutant

A mutant survives when the existing test cases do not detect the introduced change.

7.7 Equivalent Mutant

An equivalent mutant is a mutant that behaves in the same way as the original program and therefore cannot normally be detected through test cases.

8. Killed Mutant

Consider an ATM system.

The original condition is:

```
if balance >= amount
    Allow withdrawal
else
    Reject withdrawal
```

Suppose:

Balance = ₹10,000

Withdrawal = ₹10,000

The original condition becomes:

10000 >= 10000

This is true.

Therefore, withdrawal is allowed.

Now create a mutant by changing:

>=

to:

>

The mutant becomes:

if balance > amount

For the same test:

10000 > 10000

This is false.

The mutant rejects the withdrawal.

The test case detects this difference.

Therefore:

Mutant = Killed

This demonstrates how a test case can detect a small change in program logic.

9. Survived Mutant

Consider the same ATM example.

Suppose the test suite only checks:

- ₹5,000
- ₹7,000
- ₹8,000

For these values, the original condition and the mutated condition may give the same result.

Therefore, the test cases do not detect the mutation.

The mutant remains undetected.

Therefore:

Mutant = Survived

A surviving mutant indicates that the test suite may need additional test cases.

This is one of the most useful outcomes of mutation testing because it helps developers identify weaknesses in their testing.

10. Mutation Operators

Mutation operators are rules used to create small changes in a program.

They represent different types of possible faults.

10.1 Arithmetic Operator Mutation

An arithmetic operator is changed.

Original:

$$a + b$$

Mutant:

$$a - b$$

Mutation:

$$+ \rightarrow -$$

Other examples include:

$$* \rightarrow /$$
$$+ \rightarrow *$$
$$- \rightarrow +$$

10.2 Relational Operator Mutation

Comparison operators can be changed.

Original:

$$x > 10$$

Mutant:

$$x \geq 10$$

Other examples include:

$$> \rightarrow \geq$$
$$< \rightarrow \leq$$
$$== \rightarrow !=$$

10.3 Logical Operator Mutation

Logical operators can be changed.

Original:

$$A \text{ AND } B$$

Mutant:

A OR B

Mutation:

AND → OR

10.4 Conditional Mutation

Conditions can be modified.

Original:

if age >= 18

Mutant:

if age > 18

This type of mutation is useful for testing boundary conditions.

10.5 Value Mutation

Constants or values can be changed.

Original:

age = 18

Mutant:

age = 19

This allows the test suite to be checked against changes in important values.

11. Example of Mutation Operators

Consider:

if marks >= 40
 Pass

Different mutants can be created.

Mutant 1

if marks > 40

Mutant 2

if marks == 40

Mutant 3

if marks < 40

These mutations change the behavior of the condition.

A strong test suite should contain suitable test cases that can detect these changes.

12. Mutation Testing Process

Mutation testing follows a systematic process.

Step 1 – Create Original Program

A correct program is prepared.

Step 2 – Create Test Cases

Test cases are written to check the program.

Step 3 – Generate Mutants

Small changes are introduced using mutation operators.

Step 4 – Execute Test Cases

The same test suite is run against each mutant.

Step 5 – Analyze Results

Each mutant is classified as:

- Killed
- Survived
- Equivalent

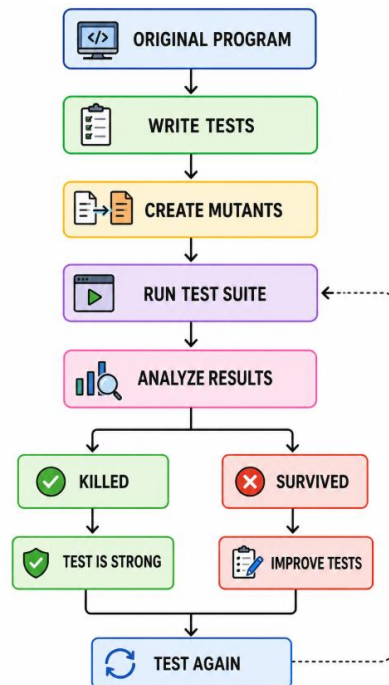
Step 6 – Improve Test Cases

If important mutants survive, additional test cases are created.

Step 7 – Repeat

Mutation testing can be performed again to evaluate the improved test suite.

13. Mutation Testing Workflow Diagram



This workflow shows that mutation testing is not simply about generating mutants. Its main purpose is to use the results to improve the test suite.

14. Mutation Score

Mutation score is a measurement used to determine how effective a test suite is at detecting mutants.

The formula is:

$$\text{Mutation Score} = \frac{\text{Killed Mutants}}{(\text{Total Mutants} - \text{Equivalent Mutants})} \times 100$$

Example

Suppose:

- Total mutants = 10
- Killed mutants = 8
- Equivalent mutants = 1

Then:

$$\begin{aligned} \text{Mutation Score} &= \frac{8}{(10-1)} \times 100 \\ &= 88.89\% \end{aligned}$$

Therefore:

Mutation Score = 88.89%

A higher mutation score generally indicates that the test suite can detect more of the introduced changes.

However, the actual quality of testing also depends on which mutants survive and how important those faults are.

15. Real-Time Example 1 – ATM Banking System

An ATM system contains a withdrawal condition:

```
if balance >= amount
    Allow withdrawal
else
    Reject withdrawal
```

Suppose:

Balance = ₹10,000

Withdrawal = ₹10,000

The original program evaluates:

$10000 \geq 10000$

The result is true.

Therefore:

Withdrawal is allowed.

Now create a mutant:

```
if balance > amount
```

The mutant evaluates:

$10000 > 10000$

The result is false.

Therefore:

Withdrawal is rejected.

The test case detects the difference.

Result:

Mutant = Killed

This example demonstrates the importance of **boundary value testing**.

The exact value of ₹10,000 is important because it is the boundary between the two conditions.

If the test suite only tested smaller withdrawal amounts, the mutation could survive.

16. Real-Time Example 2 – Online Shopping

Consider an online shopping system.

Suppose a website provides a **10% discount when the purchase amount is ₹5,000 or more**.

Original condition:

```
if amount >= 5000
    Give 10% discount
```

Suppose the purchase amount is exactly:

₹5,000

The original program checks:

$5000 \geq 5000$

The result is true.

Therefore:

10% discount is given.

Now create a mutant:

```
if amount > 5000
```

For ₹5,000:

$5000 > 5000$

The result is false.

Therefore:

No discount is given.

The test case detects this difference.

Result:

Mutant = Killed

This example demonstrates how mutation testing can test:

- Business rules
- Discount conditions
- Boundary values
- Conditional logic

17. Equivalent Mutants

Sometimes a mutation is introduced but the behavior of the program does not change.

Such a mutant is called an **equivalent mutant**.

For example, suppose a change is made inside a section of code that does not affect the final output.

The original program and mutant produce the same result.

Therefore, the test cases cannot detect a difference.

Equivalent mutants are difficult because they cannot normally be killed through normal test execution.

They are generally excluded from the mutation score calculation.

18. Advantages of Mutation Testing

Mutation testing provides several advantages.

1. Checks Test Quality

It helps determine whether test cases can actually detect faults.

2. Finds Weak Test Cases

Surviving mutants can reveal weaknesses in the test suite.

3. Identifies Missing Test Cases

It can show where additional test cases are required.

4. Improves Software Reliability

Better test cases can detect more faults.

5. Supports Boundary Testing

It is useful for testing conditions close to important boundaries.

6. Supports Automated Testing

Modern tools can automatically generate and execute mutants.

7. Provides a Measurement

Mutation score gives a numerical indication of test effectiveness.

19. Limitations of Mutation Testing

Although mutation testing is useful, it has some limitations.

1. Time Consuming

Large programs can generate a large number of mutants.

2. Computationally Expensive

Each mutant may require execution of a large test suite.

3. Equivalent Mutants

It can be difficult to identify mutants that behave exactly like the original program.

4. Large Number of Mutants

Large applications may produce hundreds or thousands of mutants.

5. Requires Automation

Manual mutation testing is difficult for large software systems.

Therefore, mutation testing should be used carefully and efficiently, especially for large applications.

20. Mutation Testing Tools

Mutation testing can be performed using specialized tools.

Java

PIT Mutation Testing

PIT can automatically create mutants, execute tests, and produce mutation reports.

Python

MutPy

MutPy can be used to perform mutation testing on Python programs.

JavaScript and .NET

Stryker

Stryker supports mutation testing for modern programming environments.

These tools can:

- Generate mutants
- Execute test cases
- Detect killed mutants
- Identify surviving mutants
- Generate reports
- Calculate mutation scores

Automation makes mutation testing more practical for real software projects.

21. Code Coverage vs Mutation Testing

Code coverage and mutation testing measure different things.

Code Coverage

“How much of my code was executed by my test cases?”

For example, 90% code coverage means that the tests executed a large portion of the measured code.

Mutation Testing

“Can my test cases detect changes or faults in the code?”

Code coverage tells us whether the code was executed, while mutation testing tells us whether our tests can detect faults in that code.

22. Applications of Mutation Testing

Mutation testing can be applied in different areas.

Banking

- ATM systems
- Payment processing
- Transaction validation
- Financial calculations

E-Commerce

- Discount systems
- Shopping carts
- Payment systems
- Order processing

Healthcare

- Medical calculations
- Patient management
- Healthcare applications

Automotive

- Vehicle control systems
- Embedded software
- Safety-related software

Mobile Applications

- Login
- Registration
- Payment
- Input validation

Web Applications

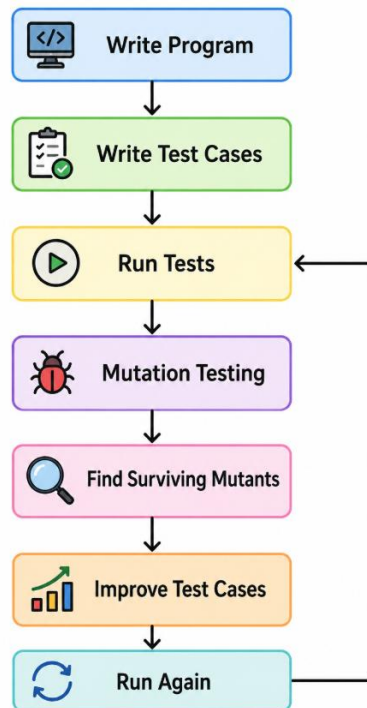
- Authentication
- Authorization
- Business logic
- Form validation

Mutation testing is especially useful when software reliability and correctness are important.

23. Mutation Testing in Software Development

Mutation testing can be included in the software development and testing process.

The workflow can be represented as:



24. Simple Overall Example

Consider a simple addition function:

```
int add(int a, int b)
{
    return a + b;
}
```

Test case:

add(5, 3)

Expected result:

Now create a mutant by changing:

$$a + b$$

to:

$$a - b$$

The mutant produces:

$$5 - 3 = 2$$

The test expects:

$$8$$

Therefore, the test fails.

The test has detected the mutation.

Result:

Mutant = Killed

25. Why is Mutation Testing Useful?

The main purpose of mutation testing is to answer one important question:

“Are my test cases good enough?”

Mutation testing does not create real bugs for users. Artificial changes are created only for evaluating the test suite.

If many important mutants survive, it may indicate:

- Missing test cases
- Weak assertions
- Missing boundary conditions
- Untested business rules
- Incomplete input combinations

If most important mutants are killed, the test suite provides stronger evidence that it can detect many types of faults.

Therefore, mutation testing helps improve the quality of software testing.

26. Points To Remember

The most important points are:

1. Mutation testing checks the **quality of test cases**.
2. A changed program is called a **mutant**.
3. The small change is called a **mutation**.
4. A detected mutant is **killed**.
5. An undetected mutant **survives**.
6. An equivalent mutant behaves like the original program.
7. Mutation score measures test effectiveness.
8. Surviving mutants can help identify weak tests.
9. Mutation testing can improve boundary testing.
10. Mutation testing is useful but computationally expensive.

The complete concept can be remembered as:

Original Program → Create Mutant → Run Test → Killed/Survived → Improve Test

27. Conclusion

Mutation testing is an important software testing technique used to evaluate the effectiveness of test cases.

It starts with a correct program and introduces small intentional changes to create mutants. The existing test cases are then executed against these mutants.

If the test cases detect the changes, the mutants are killed. If the test cases fail to detect the changes, the mutants survive.

Surviving mutants help developers identify weaknesses in their test suites. Mutation score provides a useful measurement of how effectively the test cases detect mutations.

Mutation testing is especially useful for checking boundary conditions, business rules, logical conditions, and automated test suites. Although it requires additional time and computational resources, it can provide valuable information beyond traditional code coverage.

The main idea can be summarized as:

Traditional testing tests the software, while mutation testing also tests how effective our tests are.

Therefore, mutation testing helps developers create stronger test suites and build more reliable software.

